Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251230**
Nama Lengkap	**Okky Alexander**
Minggu ke / Materi	**11 / Tipe Data List**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

MATERI SIFAT-SIFAT LIST

List dalam Python merupakan kumpulan nilai yang disimpan di bawah satu nama variabel. Berbeda dengan String yang hanya menyimpan rangkaian karakter, list jauh lebih fleksibel karena mampu menampung berbagai jenis data sekaligus — mulai dari karakter, bilangan bulat (integer), bilangan desimal (float), hingga list lain di dalamnya. Penulisannya menggunakan tanda kurung siku [], misalnya:

```
nilai_ujian = [80,75,70,90,81,84,92,71,65,80,70]

nama_pahlawan = ['Sukarno', 'Diponegoro', 'Jend. Sudirman', 'Cut Nya Dhien']

nilai_campuran = ['Javascript', 20, 34.4, True]

list_dalam_list = [23, [22, 20], 45]
```

Salah satu perbedaan mendasar antara list dan String terletak pada sifatnya. List bersifat mutable, artinya nilai di dalamnya dapat diubah langsung setelah didefinisikan. Sebaliknya, String bersifat immutable — nilainya tidak bisa dimodifikasi secara langsung. Sebagai ilustrasi, mengubah elemen list pada indeks tertentu berjalan tanpa masalah, namun mencoba mengubah karakter pada String akan langsung memunculkan TypeError.

```
data = [10, 20, 30, 40]

data[0] = 50  # berhasil → [50, 20, 30, 40]

nama = 'Antonius Rachmat'

nama[0] = 'Z'  # error! String tidak bisa diubah langsung
```

Perbedaan lainnya menyangkut cara Python mengelola memori. Apabila dua variabel String memiliki nilai yang identik, Python akan mengarahkan keduanya ke objek yang sama di memori. Namun pada list, dua variabel dengan isi yang persis sama tetap dianggap sebagai objek yang berbeda. Hal ini bisa dibuktikan menggunakan operator is:

```
>>> a = 'banana'

>>> b = 'banana'

>>> a is b

True  # merujuk objek yang sama
```

```
>>> a = [1, 2, 3]

>>> b = [1, 2, 3]

>>> a is b

False  # merujuk objek yang berbeda
```

MATERI MENGAKSES DAN MENGUBAH ISI LIST

Untuk mengambil nilai dari sebuah list, kita cukup menyebutkan nama list diikuti indeks elemen yang diinginkan dalam kurung siku. Python juga mendukung indeks negatif, di mana -1 merujuk pada elemen terakhir, -2 merujuk pada elemen kedua dari belakang, dan seterusnya. Selain mengakses satu elemen, kita pun bisa mengambil beberapa elemen sekaligus menggunakan teknik slicing dengan format [start:end].

Nilai dalam list juga dapat diperbarui kapan saja, baik untuk satu elemen maupun beberapa elemen sekaligus menggunakan rentang indeks. Perhatikan contoh berikut:

```
thislist = ["apple", "banana", "cherry"]

print(thislist[1])     # mengakses indeks ke-1 → banana

print(thislist[-1])    # mengakses dari belakang → cherry

print(thislist[2:5])   # slicing dari indeks 2 sampai 4 → ['cherry']

print(thislist[:4])    # slicing dari awal hingga indeks 3 → semua elemen

thislist[1] = "jeruk"  # mengganti "banana" menjadi "jeruk"

print(thislist)        # → ['apple', 'jeruk', 'cherry']

thislist2 = ["apple", "banana", "cherry", "orange", "kiwi", "mango"]

thislist2[1:3] = ["blackcurrant", "watermelon"]  # mengganti 2 elemen sekaligus

print(thislist2)       # → ['apple', 'blackcurrant', 'watermelon', 'orange', 'kiwi', 'mango']
```

Dengan kata lain, indeks tidak hanya berguna untuk membaca elemen, tetapi juga untuk memodifikasi isi list secara presisi — baik satu per satu maupun dalam satu blok rentang tertentu.

MATERI FUNGSI-FUNGSI UNTUK LIST

List di Python dapat digabungkan menggunakan operator + dan diulang menggunakan operator *. Operator + akan menyatukan dua list menjadi satu list baru, sementara operator * akan menduplikasi isi list sebanyak jumlah yang ditentukan.

```
thislist = [1,2,3,4,5]

thislist2 = [6,7,8]

listbaru = thislist + thislist2   # menggabungkan dua list

listbaru2 = thislist * 2          # mengulang isi list sebanyak 2 kali

print(thislist)   # → [1, 2, 3, 4, 5]

print(thislist2)  # → [6, 7, 8]

print(listbaru)   # → [1, 2, 3, 4, 5, 6, 7, 8]

print(listbaru2)  # → [1, 2, 3, 4, 5, 1, 2, 3, 4, 5]
```

Method Bawaan List

Python menyediakan sejumlah method khusus untuk memanipulasi list:

append() — Menambahkan elemen baru di akhir list dan memperlakukannya sebagai satu objek utuh.

```
nama = ['kuncoro', 'felix', 'ryan', 'yuan']

nama.append(['tejo','ujo'])  # menambahkan list sebagai satu elemen
```

extend() — Menambahkan elemen baru di akhir list, namun setiap item diperlakukan sebagai elemen individual.

```
nama.extend(['tejo','ujo'])  # menambahkan 'tejo' dan 'ujo' sebagai dua elemen terpisah
```

sort() — Mengurutkan elemen-elemen dalam list.

pop() — Menghapus elemen berdasarkan indeks tertentu sekaligus mengembalikan nilainya.

del — Menghapus elemen berdasarkan indeks tanpa mengembalikan nilainya.

remove() — Menghapus elemen berdasarkan nilai yang dicari, bukan indeksnya.

reverse() — Membalik urutan seluruh elemen dalam list.

Fungsi Built-in untuk List

Selain method di atas, Python juga menyediakan fungsi bawaan yang dapat digunakan langsung pada list:

Fungsi	Kegunaan
min()	Mencari nilai terkecil dalam list
max()	Mencari nilai terbesar dalam list
sum()	Menjumlahkan seluruh elemen dalam list
len()	Menghitung jumlah elemen dalam list

Contoh:

```
angka = [5, 3, 90, 1]

min(angka)  # → 1

max(angka)  # → 90

sum(angka)  # → 99

len(angka)  # → 4
```

List Comprehension

List comprehension adalah cara penulisan operasi pada list yang lebih ringkas dan efisien dalam satu baris kode. Berikut beberapa contoh penggunaannya:

```
nama = ["anton", "kuncoro", "buntoro", "juworo", "margono"]

# Menyaring elemen yang mengandung substring "or"

newlist = [x for x in nama if "or" in x]

# Mengubah semua elemen menjadi huruf kapital

newlist = [x.upper() for x in nama]
```

Mengambil elemen yang panjangnya lebih dari 6 karakter

newlist = [x for x in nama if len(x) > 6] Dengan list comprehension, proses penyaringan, transformasi, maupun seleksi elemen dapat dilakukan secara lebih singkat dibandingkan menggunakan perulangan for biasa.

MATERI LIST SEBAGAI PARAMETER FUNGSI

Ketika sebuah list dikirimkan sebagai argumen ke dalam fungsi, sifat mutable-nya tetap berlaku. Artinya, setiap perubahan yang dilakukan terhadap list di dalam fungsi akan secara langsung memengaruhi nilai list aslinya di luar fungsi. Hal ini berbeda dengan tipe data seperti integer atau string yang tidak berubah meski dimodifikasi di dalam fungsi.

```
def ubah(data):
    data[0] = "anton"


data = ["a", "b", "c"]
ubah(data)
print(data)  # → ['anton', 'b', 'c']
```

Terlihat bahwa nilai data[0] berubah menjadi "anton" meskipun perubahannya terjadi di dalam fungsi, bukan di luar.

Keuntungan Menggunakan List sebagai Parameter

Menggunakan list sebagai parameter fungsi membawa sejumlah manfaat praktis:

1. **Fleksibilitas** — Fungsi mampu memproses beragam jenis data karena list dapat menampung berbagai tipe sekaligus.
2. **Reusabilitas** — Fungsi yang sama dapat digunakan berulang kali dengan list yang berbeda-beda.
3. **Pembagian Tugas** — List memudahkan pemecahan masalah kompleks menjadi bagian-bagian yang lebih kecil dan terstruktur.
4. **Pengorganisasian Data** — List membantu mengelompokkan data secara rapi sebelum diolah lebih lanjut oleh fungsi.

Contoh: Menghapus Elemen Pertama List

Berikut ilustrasi perbedaan antara fungsi yang **mengembalikan nilai baru** dengan yang **mengubah list secara langsung**:

python

```
def hapus(data):

    return data[1:]  # mengembalikan list baru tanpa elemen pertama

hapus(data)   # → ['b', 'c'], tapi data asli belum berubah

print(data)   # → ['anton', 'b', 'c']

data = hapus(data)  # hasil dikembalikan ke variabel data

print(data)       # → ['b', 'c']
```

Perlu diperhatikan bahwa fungsi hapus() di atas tidak mengubah list secara langsung, melainkan menghasilkan list baru melalui slicing. Agar perubahan tersimpan, hasil fungsi harus ditampung kembali ke dalam variabel, seperti data = hapus(data).

Source Github : https://github.com/tuangkeman/71251230_Okky.git

LATIHAN 11.1

```
import re
def tt(data):
    data = sorted(list(map(int,data)), reverse=True)
    return data[:3]

nilai = input("Masukan bilangan list :")
nilai = re.findall(r'\d+', nilai)

hasil = tt(nilai)
print("List inputan    :", list(map(int,nilai)))
print("3 nilai terbaik:", list(map(int,hasil)))
```

Gambar 11.1: kode mencari bilangan terbesar

Kode ini terdiri dari sebuah fungsi bernama tt(data) yang bertugas mencari 3 bilangan terbesar dari sebuah list. Di dalam fungsi tersebut, setiap elemen data dikonversi terlebih dahulu menjadi tipe integer menggunakan map(int, data), karena data yang masuk masih berupa string. Hasil konversi kemudian diubah menjadi list, lalu diurutkan dari nilai terbesar ke terkecil menggunakan sorted(..., reverse=True). Setelah terurut, fungsi mengembalikan tiga elemen pertama melalui data[:3] yang merupakan 3 bilangan tertinggi.

Pada bagian input, program meminta pengguna mengetikkan sekumpulan angka melalui input(). Angka-angka tersebut kemudian diekstrak menggunakan re.findall(r'\d+', nilai) yang memanfaatkan **Regular Expression**. Pola \d+ berarti "satu atau lebih digit angka", sehingga program mampu mengenali angka meskipun dipisahkan oleh spasi, koma, titik, atau karakter lain apa pun. Hasilnya adalah list berisi string angka seperti ['45', '92', '78'].

Terakhir, fungsi tt() dipanggil dengan nilai sebagai argumennya dan hasilnya disimpan dalam variabel hasil. Saat menampilkan output, map(int, ...) digunakan kembali untuk mengonversi elemen-elemen list dari string menjadi integer agar tampilannya lebih rapi, sehingga yang tercetak adalah angka bulat, bukan string.

LATIHAN 11.2

```
b = []
while True:
    m = input("Masukkan bilangan atau done untuk selesai: ")
    if m.lower() == "done":
        break
    try:
        a = float(m)
        b.append(a)
    except ValueError:
        print("Input tidak valid, masukkan bilangan atau done")

if len(b) > 0:
    r = sum(b) / len(b)
    print()
    print("Bilangan yang dimasukkan:", b)
    print("Rata-rata              :", r)
else:
    print("Tidak ada bilangan yang dimasukkan.")
```

Gambar 11.2: kode rata rata

Program dimulai dengan membuat list kosong b yang nantinya akan menampung semua bilangan yang dimasukkan pengguna. Kemudian program masuk ke dalam perulangan while True yang akan terus berjalan tanpa henti sampai ada kondisi tertentu yang menghentikannya.

Di dalam perulangan, program meminta input dari pengguna dan menyimpannya ke variabel m. Setiap input yang masuk langsung diperiksa — jika pengguna mengetik "done" (dengan huruf besar maupun kecil, karena menggunakan .lower()), maka perulangan langsung dihentikan menggunakan break. Jika bukan "done", program mencoba mengonversi input tersebut menjadi bilangan desimal menggunakan float(m) dan menambahkannya ke list b melalui append(). Proses konversi ini dibungkus dalam blok try-except untuk mengantisipasi kemungkinan pengguna memasukkan karakter yang tidak bisa dikonversi menjadi angka — jika hal itu terjadi, program tidak akan berhenti dengan error, melainkan menampilkan pesan peringatan dan kembali meminta input.

Setelah perulangan selesai, program memeriksa apakah list b memiliki isi menggunakan len(b) > 0. Jika ada bilangan yang tersimpan, rata-rata dihitung dengan membagi total seluruh elemen sum(b) dengan jumlah elemen len(b), lalu hasilnya disimpan ke variabel r dan ditampilkan bersama list bilangan yang telah dimasukkan. Sebaliknya, jika pengguna langsung mengetik "done" tanpa memasukkan satu pun angka, program akan menampilkan pesan bahwa tidak ada bilangan yang dimasukkan, sekaligus menghindari terjadinya pembagian dengan nol.

LATIHAN 11.3

```
import re
def ku(n):
    try:
        with open(f"Minggu 11/{n}", 'r') as f:
            t = f.read()
        k = re.findall(r'\b[a-zA-Z]+\b', t.lower())
        u = sorted(set(k))
        print(f"Total semua kata : {len(k)}")
        print(f"Total kata unik : {len(u)}")
        print()
        print(f"Daftar kata unik:")
        for i, kata in enumerate(u, 1):
            print(f"{i:3}. {kata}")
    except FileNotFoundError:
        print(f"File '{n}' tidak ditemukan.")
nf = input("Masukkan nama file: ")
ku(nf)
```

Gambar 11.3: kode kata unik

Program dimulai dengan mengimpor modul re yang digunakan untuk memproses teks menggunakan Regular Expression. Kemudian didefinisikan sebuah fungsi bernama ku(n) di mana parameter n mewakili nama file yang akan dibaca.

Di dalam fungsi, program mencoba membuka file yang berada di folder "Minggu 11/" menggunakan open(f"Minggu 11/{n}", 'r'). Nama file digabungkan dengan path folder secara otomatis menggunakan f-string, sehingga pengguna cukup mengetik nama filenya saja. Seluruh isi file kemudian dibaca sekaligus menggunakan .read() dan disimpan ke variabel t.

Setelah file terbaca, re.findall(r'\b[a-zA-Z]+\b', t.lower()) digunakan untuk mengekstrak semua kata dari teks. Sebelum diekstrak, teks diubah dulu menjadi huruf kecil menggunakan .lower() agar kata seperti "Teknologi" dan "teknologi" dianggap sama. Pola \b[a-zA-Z]+\b memastikan hanya kata-kata yang diambil, sementara angka dan tanda baca diabaikan. Hasilnya disimpan ke variabel k berupa list semua kata.

Selanjutnya, set(k) digunakan untuk menghilangkan kata-kata yang muncul lebih dari satu kali, sehingga hanya tersisa kata unik. Hasil tersebut kemudian diurutkan secara alfabetis menggunakan sorted() dan disimpan ke variabel u. Program lalu menampilkan total keseluruhan kata, total kata unik, serta daftar lengkap kata unik beserta nomornya menggunakan enumerate().

Jika file yang dimasukkan tidak ditemukan, blok except FileNotFoundError akan menangkap error tersebut dan menampilkan pesan bahwa file tidak ditemukan, sehingga program tidak berhenti secara mendadak. Di bagian akhir, program meminta input nama file dari pengguna melalui input() dan menyimpannya ke variabel nf, kemudian memanggil fungsi ku(nf) untuk menjalankan seluruh proses.